

From Geometric Recovery to Causal Validation: A Reproducible Audit of Sparse Autoencoder Features, from Superposition Geometry to Causal Inertness

Mohamed Abdesslem Bal
Independent Researcher, Algiers, Algeria
mohamed.bal@devupdz.com

Code: <https://github.com/mohamed-bal/superposition-to-monosemanticity>
<https://github.com/mohamed-bal/sae-causal-audit>

July 2026

Abstract

Sparse autoencoders (SAEs) are the de facto standard for decomposing superposed neural representations into interpretable features, and the field’s evaluation practice relies predominantly on correlational recovery metrics—cosine similarity between ground-truth (or probed) directions and learned decoder atoms. We show, in a fully controlled setting where ground truth is known, that this practice conflates two distinct empirical claims: decoder-geometry alignment and encoder-activation behavior. We first reproduce, from scratch and with multi-seed statistics, the superposition phase diagram of Elhage et al. (2022), identifying along the way a convergence artifact at high sparsity and a previously under-described *diffuse sharing* regime at extreme overcompleteness. We then reproduce the TopK-versus- L_1 SAE comparison of Gao et al. (2024), including direct quantitative evidence of L_1 shrinkage (activation-magnitude refinement recovers 91.2% of the L_1 SAE’s reconstruction gap while shifting magnitudes +22.5%, versus -0.05% for TopK). Our central result is causal: subjecting every correlationally recovered feature to ablation and steering interventions, we find that up to 77% of features passing a standard recovery bar (cosine ≥ 0.90) in a degraded SAE—and 9% in a well-trained one—are *causally inert*: the matched atom never fires when the feature is present, including matches at cosine ≈ 1.000 . We package the methodology as **sae-causal-audit**, a model-agnostic, protocol-typed instrument with a hash-verified deterministic reproduction pipeline, and re-auditing the setting under it refines the finding twice: causal inertness decomposes by *cause* into structural inertness (traceable to antipodal-pair superposition geometry and present in good SAEs) and competitive inertness (a TopK-selection pathology of degraded SAEs), and by *direction* into read-inertness and write-inertness, which five antipodal pairs dissociate completely—features that are unmonitorable yet steerable through the same atom, with steering specificities of 143–310 attached to ablation effects of exactly zero. We additionally document why byte-exact cross-platform reproducibility is unavailable by construction for this class of workload, and propose reporting reproducibility as a stack of claims with explicit scopes.

1 Introduction

Mechanistic interpretability’s current toolkit rests on a specific pipeline: hypothesize that networks represent more features than they have dimensions by packing them into almost-orthogonal directions (*superposition* [6]), train a sparse autoencoder to un-mix those directions into an overcomplete

dictionary [2], and evaluate the dictionary by how well its atoms align, geometrically, with known or probed feature directions. This pipeline scales: Anthropic extracted millions of features from a production model [13], and OpenAI and Google DeepMind have published comparable open-weight SAE suites [7, 10].

Recovering *a* sparse, plausible-looking decomposition, however, is not the same as demonstrating that the decomposition is the one the model actually uses. No interpretability method today gives a certified, complete account of a model’s internal computation [1]. The evaluation gap this paper targets is narrower and precisely characterizable: the field’s dominant recovery metric—unsigned cosine similarity between a target direction and its best-matching *decoder* atom—measures reconstruction geometry, while whether an atom ever *fires* for a given input is governed by the *encoder* and its selection nonlinearity (TopK, ReLU). These are two different empirical claims, and only an intervention can tell them apart.

We work in the most favorable setting that can be constructed for the recovery pipeline: a small synthetic model in which ground truth is fully known and controlled. This choice is deliberate. The toy setting is what makes rigorous causal validation *possible*; it is also exactly what makes every absolute number reported here a calibration point for the easy case, not a benchmark for a production model.

Contributions.

1. **A multi-seed reproduction of the superposition phase diagram** [6], with two additions: a documented convergence artifact at high sparsity (under-training masquerading as superposition, resolved by sparsity-aware step budgets), and an under-described third geometric regime—*diffuse sharing*—at extreme overcompleteness, in which every feature is substantially represented ($\|W_i\|^2 \approx 1.40$) while feature dimensionality reads $D_i \approx 0.08$ (Section 4).
2. **A from-scratch reproduction of the TopK-vs- L_1 Pareto comparison** [7], including a direct activation-refinement test that quantifies L_1 shrinkage [14] rather than asserting it (Section 5).
3. **A causal validation battery** (ablation and sign-correct steering, propagated through the model’s actual output stage) applied to every correlationally recovered feature, establishing that correlational recovery and causal efficacy diverge by a measured margin: 17 of 22 matched features causally inert in a degraded SAE, 2 of 22 in a well-trained one, with inert matches at cosine up to 0.9998 (Sections 6 and 9).
4. **sae-causal-audit**, a model-agnostic instrument packaging the battery behind two structural protocols (any object that can **encode**, **decode**, and expose its dictionary is auditable, including `sae_lens.SAE`), with interface design that makes a documented sign-handling bug class unrepresentable (Section 7).
5. **A two-axis taxonomy of causal inertness**. By *cause*: structural inertness, generated systematically by antipodal-pair superposition geometry under a positive-pass encoder and *not* removed by better SAE training; versus competitive inertness, a TopK-competition pathology of degraded dictionaries that is simultaneously the locus of numerical boundary instability. By *direction*: read-inertness (ablation-based monitoring is blind) versus write-inertness (steering is impotent), which antipodal pairs dissociate completely (Section 10).
6. **Reproducibility as a stack of claims**. We document, through a three-layer debugging episode on CI infrastructure, why byte-exact reproducibility across platforms is unavailable by construction for floating-point training workloads, and we propose—and implement—a

two-tier guarantee: SHA-256 byte-exactness within one pinned environment, semantic equality ($\text{rtol} = 10^{-4}$, boundary-sensitive integer counts within ± 1) everywhere (Section 8).

2 Background and Related Work

Superposition. Individual neurons in trained networks frequently activate for several unrelated concepts—*polysemanticity*—which is why reading meaning off single neurons has never worked reliably as a general strategy. Elhage et al. [6] propose a two-part explanation: the *linear representation hypothesis* (features correspond to directions in activation space, not basis coordinates) and the *superposition hypothesis* (when features are sparse—rarely simultaneously active—a model benefits from representing more features than dimensions by assigning them almost-orthogonal directions, accepting rare interference in exchange for capacity). The geometric justification is the Johnson–Lindenstrauss phenomenon [8]: a d -dimensional space admits exponentially more than d near-orthogonal directions once “exactly orthogonal” is relaxed to “inner product bounded near zero.” Under symmetric two-feature competition for one dimension, the stable configuration found by gradient descent is the *antipodal pair* $W_i = -W_j$; with more features, higher-order polytope-like arrangements emerge, structurally analogous to the Thomson problem of packing mutually repelling charges on a sphere. This paper reproduces the two-feature antipodal case exactly and the large-scale phase structure statistically; full polytope geometry (3–8 feature models in isolation) is out of scope.

Dictionary learning and SAEs. Recovering individual features from a superposed mixture is a sparse coding problem with a history predating the interpretability boom: Olshausen and Field [11] explained simple-cell receptive fields as emergent sparse codes over natural images. The interpretability instantiation [2, 13] trains the same mathematical object on transformer activations. Exact L_0 (cardinality) minimization is NP-hard; the classical convex relaxation penalizes the code’s L_1 norm, at the documented cost of *shrinkage* [14]: every active coefficient is biased toward zero, correct ones included. Gao et al. [7] remove the penalty entirely and enforce sparsity structurally via a TopK selection in the encoder, with an auxiliary loss resurrecting “dead” atoms; subsequent variants include Gated SAEs [12] and BatchTopK [3]. Published open-weight suites now exist for GPT-2 and Gemma-2 [7, 10].

Evaluation practice and its gap. Recovery is conventionally scored correlationally: a ground-truth (or linear-probe) direction counts as recovered if its best-matching decoder atom exceeds a cosine-similarity bar, typically taken unsigned because a feature and its exact negation are equally good correlational matches. Causal evaluation—ablation, activation patching, steering—is standard for *circuits* but is not routinely applied per-feature to certify recovery claims. A 2024 survey [1] additionally flags an unresolved concern: increasingly capable models could develop internal structure that resists or misleads the very techniques probing them. Our results bear on this indirectly (Section 12): if decoder geometry can diverge this far from encoder behavior *without any adversarial pressure*, purely as an ordinary training-dynamics artifact, the space of “geometrically plausible, causally misleading” solutions available under actual optimization pressure is at least as large.

Firing failures and proxy-metric gaps. The closest prior work to our central finding is *feature absorption* [4]: on production LLM SAEs, Chanin et al. show that seemingly monosemantic latents fail to fire on inputs where their concept is plainly present, the activation having been “absorbed” into more specific latents—a failure mode caused by the sparsity objective whenever the underlying

features form a hierarchy, and not resolved by varying SAE size or sparsity. The observable symptom—an apparently recovered latent that does not activate when its feature occurs—is the same one this paper measures, and the two mechanisms are complementary rather than competing. Absorption requires hierarchical feature structure; the *structural inertness* we isolate (Section 10) arises with no hierarchy at all, from antipodal-pair superposition geometry meeting a positive-pass encoder, and our *competitive inertness* is a TopK-selection pathology of degraded dictionaries. Our setting also differs in what it can certify: absorption is detected on real models via probing-based metrics, necessarily correlational at the point of measurement, whereas our audit is interventional end to end (ablation and signed steering propagated through the model’s output) against exact ground truth—which is what exposes the read/write dissociation of Section 10.2, a decomposition invisible to firing statistics alone, since an atom that never fires for a feature can still steer it with high specificity. On the evaluation-practice side, SAEBench [9] documents at benchmark scale that gains on unsupervised proxy metrics do not reliably translate into practical performance across eight evaluation axes and 200+ SAEs; the present work contributes the per-feature, per-direction causal instrument that this broader diagnosis calls for.

3 Experimental Setup

3.1 Toy model

All experiments build on a small, fully from-scratch model (no pretrained weights, no external data; seconds on CPU):

$$h = Wx, \quad \hat{x} = \text{ReLU}(W^\top h + b), \quad (1)$$

with $W \in \mathbb{R}^{n_{\text{hidden}} \times n_{\text{features}}}$ and the interesting regime $n_{\text{hidden}} < n_{\text{features}}$: a bottleneck that forces a choice about what to represent. Inputs are synthetic and sparse: each of the n_{features} entries is independently zero with probability s (the *sparsity*) and drawn $\text{Uniform}(0, 1)$ otherwise. Training minimizes an importance-weighted reconstruction loss

$$\mathcal{L} = \sum_i I_i (x_i - \hat{x}_i)^2. \quad (2)$$

Two importance schedules answer different questions and are never mixed within an experiment:

- **Decayed importance** ($I_i = 0.9^i$) breaks symmetry so the model has a principled reason to prioritize. It produces a sharp emergent-triage finding of its own: training 32 features into an 8-dimensional bottleneck at $s = 0.95$, the optimizer does not spread capacity thinly—it fully drops the 10 lowest-importance features (columns of W collapse to near zero) and represents the remaining 22 well. Nothing hand-codes which features are dropped. This configuration (“32→8, $s = 0.95$, decayed”) is the substrate for every SAE experiment below.
- **Uniform importance** ($I_i = 1$) removes prioritization as a confound and is used exclusively for the phase diagram, isolating pure geometric structure.

3.2 Measuring superposition: feature dimensionality and its blind spot

Following Elhage et al. [6], each feature’s effective ownership of a dedicated dimension is

$$D_i = \frac{\|W_i\|^2}{\sum_j (W_i \cdot W_j)^2}. \quad (3)$$

Three exact cases, each enforced by a unit test rather than asserted: orthonormal features give $D_i = 1$; an antipodal pair gives $D_i = 0.5$ for both members; and—a subtlety with consequences for a real finding below— D_i is a *ratio* of squared norms and therefore not a magnitude detector. For $W = \begin{pmatrix} 1 & 0 & 10^{-6} \\ 0 & 1 & 10^{-6} \end{pmatrix}$, the third feature’s column norm is negligible yet its D_i is exactly 1.0, because numerator and dominant denominator term scale identically with column magnitude. **A genuinely dropped feature and a substantially represented but relatively diffuse feature can report identical D_i** ; distinguishing them requires checking $\|W_i\|^2$ directly. This check is built into the classification pipeline from the start, and Section 4.4 is the payoff.

4 The Superposition Phase Diagram

4.1 Methodology

Fixed $n_{\text{hidden}} = 20$; overcompleteness ratio $n_{\text{features}}/n_{\text{hidden}} \in \{1, 2, 4, 8\}$; sparsity $s \in \{0, 0.3, 0.6, 0.8, 0.9, 0.95, 0.99\}$; uniform importance throughout; **3 independent seeds per cell**, all numbers reported as mean $\pm$ standard deviation. Each feature is classified by final D_i : *dedicated* ($D_i > 0.9$), *superposed* ($0.1 \leq D_i \leq 0.9$), or a candidate “not represented” case ($D_i < 0.1$), which—per Section 3.2—is further split by $\|W_i\|^2$ into *truly dropped* ($\|W_i\|^2 < 0.1$) versus *diffusely shared* ($\|W_i\|^2 \geq 0.1$).

4.2 A convergence artifact, caught and documented

The first full sweep produced an impossible result at ratio = 1 ($n_{\text{features}} = n_{\text{hidden}} = 20$, where capacity suffices for a fully orthogonal solution and the correct converged answer is 100% dedicated at every sparsity): at $s = 0.95$ and 0.99 , roughly 8% and 82% of features respectively classified as superposed. There is no capacity pressure forcing superposition in this configuration, so the result was investigated rather than reported. Re-running the suspicious cells at 8,000 training steps instead of the sweep default of 1,500 fully resolved the discrepancy: dedicated fraction returns to exactly 1.00 ± 0.00 with loss ≈ 0 at every sparsity. The mechanism: at high sparsity a given feature appears in few training batches, so its gradient signal arrives rarely—convergence is *slower*, not *harder*—and a flat step budget tuned for low-sparsity cells silently under-trains the rest. The fix allocates the larger budget specifically to the affected cells. We report this in full because “looked wrong, checked why, fixed it” is the epistemic standard every other number in this paper is held to; under-training masquerading as superposition is a failure mode any replication of [6] at high sparsity should check for.

4.3 Results

Ratio = 1 (capacity-sufficient control): dedicated fraction 1.00 ± 0.00 , superposed 0.00 ± 0.00 , loss ≈ 0 , at all seven sparsity levels. With no capacity pressure the model never chooses superposition; this row is as much a pipeline sanity check as a result.

The qualitative pattern (Table 1, Figure 1) matches the theory directly. At $2\times$ overcompleteness the transition is well underway by $s = 0.3$ (74% superposed); at $4\times$ and $8\times$ it occurs later (near $s = 0.6$) and at a similar critical sparsity for both—consistent with the picture that once superposition is worth adopting at all, further overcompleteness changes *how much* is packed rather than *whether* packing begins.

Table 1: Dedicated-feature fraction (mean $\pm$ std over 3 seeds) under genuine overcompleteness. More overcompleteness requires *less* sparsity before superposition dominates. *At ratio 8, $s = 0.99$, the 0% dedicated coincides with 0% superposed: a third regime, not an omission (Section 4.4).

Sparsity s	Ratio = 2	Ratio = 4	Ratio = 8
0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
0.30	0.26 ± 0.01	1.00 ± 0.00	1.00 ± 0.00
0.60	0.02 ± 0.02	0.51 ± 0.02	0.47 ± 0.07
0.80	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
0.90	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
0.95	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
0.99	0.00 ± 0.00	0.00 ± 0.00	$0.00 \pm 0.00^*$

4.4 An unplanned third regime: diffuse sharing

The most extreme cell—ratio 8, $s = 0.99$ (160 features into 20 dimensions, 99% of inputs zeroed per sample)—reports $D_i \approx 0.08$ for every feature, below the 0.1 “candidate not-represented” threshold; the raw classification would call all 160 features dropped. Checking $\|W_i\|^2$ directly tells the opposite story: mean $\|W_i\|^2 \approx 1.40$, *larger* than a cleanly represented orthogonal feature’s norm ($\|W_i\|^2 = 1$ at the ratio-1 optimum). Every feature is substantially represented, in a highly symmetric configuration where each overlaps slightly with very many others rather than owning a dimension (dedicated) or sharing exactly one with exactly one partner (antipodal). Reconstruction loss at this cell is low (0.136 ± 0.009): the model succeeds via a third geometric strategy that the clean dedicated/antipodal/dropped trichotomy does not name. Whether this many-way diffuse packing is a documented phase or an under-described corner of the diagram is left open here; it is invisible unless $\|W_i\|^2$ is checked alongside D_i , which is why that check exists in the pipeline (Section 3.2).

5 Dictionary Learning: L_1 versus TopK

5.1 Formulations

The L_1 SAE is the classical convex relaxation of the NP-hard L_0 objective:

$$\mathcal{L}_{L_1} = \|h - \hat{h}\|_2^2 + \lambda \|f\|_1, \quad (4)$$

which inherits Lasso shrinkage [14]: every active coefficient is pushed toward zero regardless of its true optimal value, so reported activation strengths systematically under-estimate feature presence. The TopK SAE [7] removes the penalty and enforces sparsity structurally—the encoder keeps the k largest pre-activations (ReLU-rectified) and zeroes the rest, so k directly *is* the L_0 :

```
def encode(self, h):
    pre_act = (h - self.b_dec) @ self.W_enc + self.b_enc
    topk_vals, topk_idx = torch.topk(pre_act, k=self.config.k, dim=-1)
    topk_vals = torch.relu(topk_vals)
    f = torch.zeros_like(pre_act)
    f.scatter_(-1, topk_idx, topk_vals)
    return f
# loss = ((h - h_hat)**2).sum(-1).mean() # no L1 term at all
```

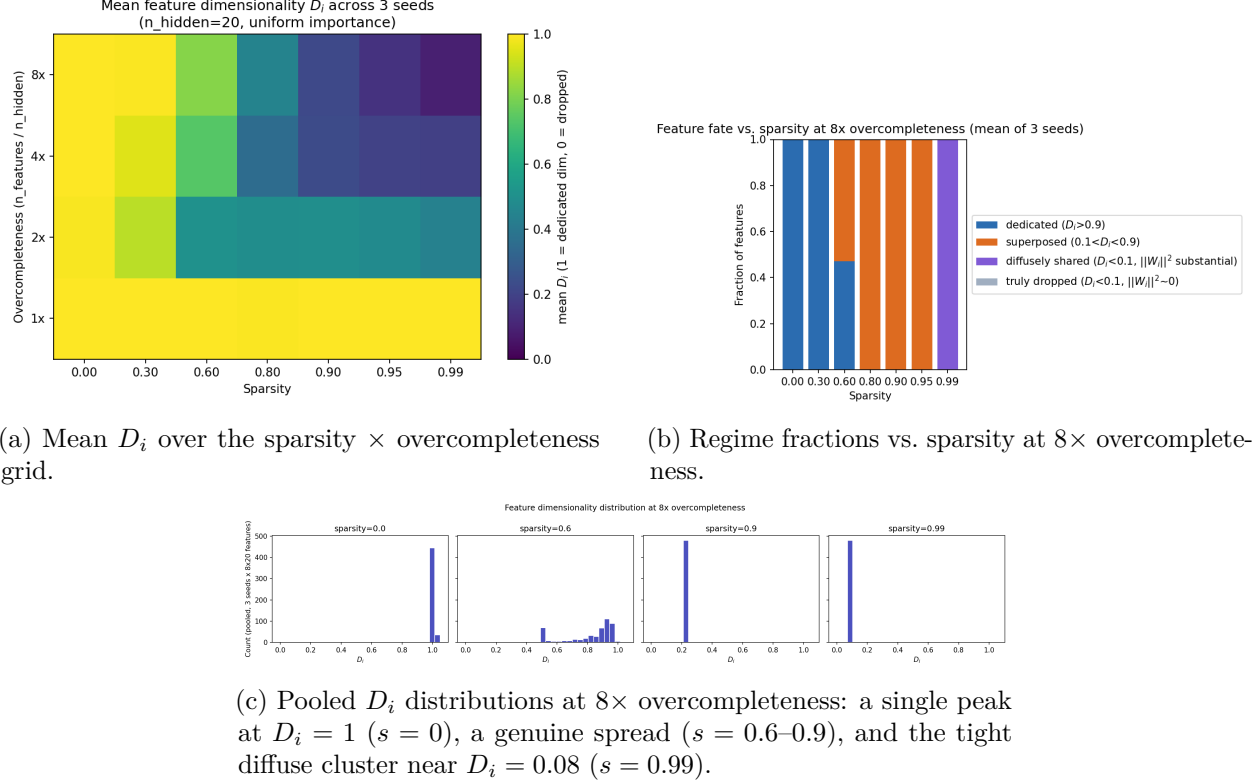


Figure 1: The superposition phase diagram under uniform importance, 3 seeds per cell.

With nothing to game by shrinking magnitudes, the only pressure on the k surviving activations is reconstruction. TopK introduces its own failure mode—atoms that stop winning the selection stop receiving gradient and “die”—mitigated here by a simplified version of the auxiliary loss of [7] (atoms silent within a rolling window receive a secondary residual-reconstruction objective). This is adequate to demonstrate the mechanism at toy scale, not a production implementation.

5.2 Pareto comparison

Both variants are trained on hidden activations of the same $32 \rightarrow 8$, $s = 0.95$, decayed-importance toy model (22 well-represented features, 10 dropped), with $16\times$ -overcomplete dictionaries (128 atoms), **3 seeds per configuration**. *Recovery precision* counts well-represented ground-truth features whose best decoder-atom match has unsigned cosine ≥ 0.90 .

Two matched- L_0 comparisons carry the result (Table 2, Figure 2a). At $L_0 \approx 4$, TopK reaches *perfect* recovery precision on the well-represented set (1.00 ± 0.00 across all seeds—every one of the 22 represented features recovered, every seed) against L_1 ’s 0.77 ± 0.11 . At $L_0 \approx 9$ – 13 , TopK’s reconstruction loss is roughly an order of magnitude lower at comparable sparsity, while precision degrades on a similar curve for both methods as L_0 grows (more simultaneously active atoms means more cross-feature interference per match). TopK dominates the Pareto front at every L_0 tested—the qualitative claim of [7], reproduced from scratch on an unrelated synthetic model.

Table 2: L_1 vs. TopK on identical superposed activations (mean $\pm$ std, 3 seeds).

Method	Hyperparam.	L_0	Recon. loss	Recovery precision
L_1	$\lambda = 0.003$	14.68 ± 0.58	0.00309 ± 0.00039	0.15 ± 0.06
L_1	$\lambda = 0.01$	10.69 ± 0.58	0.00334 ± 0.00021	0.23 ± 0.10
L_1	$\lambda = 0.03$	7.90 ± 0.73	0.00515 ± 0.00029	0.27 ± 0.07
L_1	$\lambda = 0.1$	4.42 ± 0.32	0.01427 ± 0.00074	0.77 ± 0.11
TopK	$k = 13$	12.92 ± 0.07	0.00031 ± 0.00010	0.11 ± 0.06
TopK	$k = 9$	8.99 ± 0.00	0.00060 ± 0.00009	0.23 ± 0.07
TopK	$k = 6$	6.00 ± 0.00	0.00166 ± 0.00104	0.53 ± 0.13
TopK	$k = 4$	4.00 ± 0.00	0.01064 ± 0.00210	1.00 ± 0.00

Table 3: Support-frozen magnitude refinement at matched sparsity ($L_0 \approx 4$; $k=4$ TopK vs. $\lambda=0.1$ L_1 ; held-out batch of 5,000 activations).

Method	L_0	MSE improvement	Mean active-magnitude change
L_1	4.23	91.2%	+22.5%
TopK	4.00	64.8%	−0.05% (indistinguishable from zero)

5.3 Direct evidence of shrinkage

Dominance alone does not establish *why*; it is consistent with TopK being generically better for unrelated reasons. Gao et al. propose a falsifiable mechanism (shrinkage) with a specific test, reproduced exactly here: freeze which atoms each SAE selected for a held-out batch (its support), then re-optimize *only the magnitudes* of those atoms—projected gradient descent, non-negativity constraint, decoder frozen—to directly minimize reconstruction error. If the SAE’s reported activations were already optimal given its own support, refinement cannot improve anything.

Refining the L_1 SAE’s activations closes 91% of its reconstruction gap by pushing magnitudes up nearly a quarter on average—direct quantitative evidence that the penalty suppresses true feature strengths, exactly as the Lasso analysis predicts (Table 3, Figure 2b). TopK refinement still closes some gap (refinement is better-or-equal by construction; the originals came from one gradient trajectory, not an exact solve) but its mean magnitude does not move: there is no systematic direction to correct because no penalty pushed in one. Practical consequence: an L_1 dictionary’s activation *strengths* are systematically biased low—relevant to any downstream use ranking features by magnitude, thresholding anomaly signals, or comparing strength across inputs—while TopK trades that bias for a rigid per-sample budget and the dead-atom problem.

6 From Correlation to Causation

Recovery precision of 1.00 for TopK $k=4$ is still a correlational claim: a learned decoder direction *points* at the right ground-truth feature. It is not a causal claim: that *intervening* on the direction changes behavior the way the label implies. A decoder atom can sit at the geometrically correct location without the *encoder* ever activating it when the feature is genuinely present—two failure modes hiding behind one cosine number. Closing the gap requires an intervention, not another measurement of the same representation.

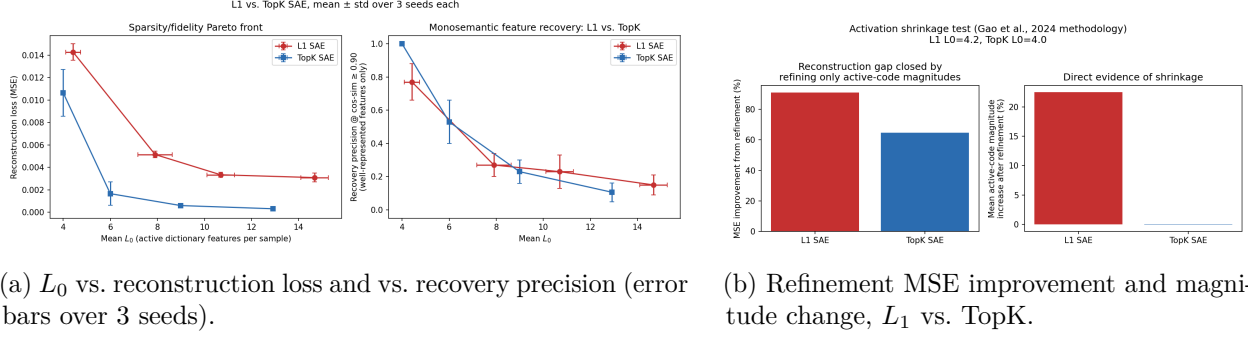


Figure 2: The L_1 /TopK comparison and the shrinkage mechanism test.

6.1 Ablation and steering

For every well-represented ground-truth feature i , matched by cosine similarity to dictionary atom j , two interventions are propagated through the toy model’s actual output stage $\hat{x} = \text{ReLU}(\hat{h}W + b)$, never a proxy:

- **Ablation.** Present inputs where *only* feature i is active; encode; zero atom j ’s code entry; decode; compare the targeted drop in feature i ’s reconstructed output against the mean absolute movement of every other feature. Their ratio is the *ablation specificity*: a causally faithful match hits its target hard and everything else barely at all; a ratio near 1 is indistinguishable from generic collateral damage.
- **Steering.** The reverse direction: inputs where feature i is *off*; force atom j to a fixed value with the correct match *sign*; measure the targeted rise.

```
f_ablated = f.clone()
f_ablated[:, atom_idx] = 0.0
x_hat_ablated = torch.relu(sae.decode(f_ablated) @ toy_model.W + toy_model.b)

targeted_drop = (x_hat_base[:, feat_idx] - x_hat_ablated[:, feat_idx]).mean()
off_target_effect = (x_hat_base - x_hat_ablated).abs()[:, other_features].mean()
specificity_ratio = targeted_drop / off_target_effect
```

A sign-convention bug, and what it demonstrates. Cosine matching takes absolute values—correctly, since a feature and its exact negation are equally good *correlational* matches—but an intervention is direction-sensitive. The first steering implementation silently assumed positive alignment and always pushed the matched atom toward +1. The test suite caught it immediately: on a sanity-check model whose true match was *anti-aligned* (dominant weight -0.82), steering “on” in the wrong direction pushed the reconstruction into a region the output ReLU clips, and the measured effect read exactly 0.0 instead of positive. The fix recovers the signed match explicitly (`best_match_indices_and_signs`) and makes the sign a required argument of `steering_effect`, so the bug cannot silently reappear through the same code path. Section 7 promotes this from a patch to an interface contract; Section 10 shows the structure that fixing it revealed.

Table 4: Causal specificity of correlationally matched features (original multi-threaded runs; see Section 9 for the deterministic re-audit).

SAE	Median abl. specificity	Min	Max	Median steer. specificity
TopK $k=4$ (precision 1.00)	168.8	0.0	2608.6	149.9
TopK $k=13$ (precision 0.11)	0.0	−902.5	163.0	21.4

6.2 Does a good correlational match predict a good causal match?

Two TopK SAEs from opposite ends of the Pareto front are audited: $k = 4$ (recovery precision 1.00; “good”) and $k = 13$ (precision 0.11; deliberately degraded, “bad”). For each of the 22 well-represented features, both interventions run against each SAE’s matched atom.

The good SAE’s median ablation specificity is 168.8—ablating the matched atom moves the target roughly 169 times more than everything else—against the bad SAE’s median of *exactly zero* (Table 4, Figure 3). Pooled correlation between cosine similarity and log-specificity: $r = 0.657$ —a real positive relationship (better correlational matches are better causal matches on average), far from $r = 1$, and the exceptions are the informative part.

6.3 Why the degraded SAE’s median is exactly zero

Seventeen of the bad SAE’s 22 matched pairs report ablation specificity = 0.0 *exactly*. Tracking `fired_frac` (the fraction of feature-present samples on which the matched atom’s code is nonzero) alongside the ratio gives an unambiguous cause: in all seventeen cases the matched atom *never fires* when the feature is presented in isolation (`fired_frac` = 0.0 over 500 samples). Ablating an already-zero code entry cannot change anything; the ratio is not small but undefined-and-reported-as-zero, for a different reason than a weak effect. One of the seventeen (feature 20) has cosine similarity 0.92—above the ≥ 0.90 recovery bar used throughout. **By the field’s correlational metric, feature 20 is a recovery success; by the causal test, the atom credited with recovering it never once activates for it.** This is 17 of 22 pairs (77%), not a dredged-up edge case. The good SAE exhibits the same mode at a lower rate: 2 of 22 matches (9%), both at cosine > 0.9999 . Even the best configuration tested is not immune to geometrically perfect, causally inert matches.

6.4 Mechanism: decoder geometry versus encoder selection

Cosine matching—here and in the literature this reproduces—compares target directions against *decoder* atoms W_{dec} , the vectors used to reconstruct activations from a code. Whether an atom *fires* is governed by the *encoder* weights and the TopK/ReLU selection, not by decoder geometry. An SAE under pressure to reconstruct well can learn a decoder atom at the geometrically correct location—useful for the reconstruction objective it is actually trained on—while the paired encoder entry essentially never wins the TopK competition for that feature, because something else usually wins first. Decoder-geometry recovery and encoder-activation recovery are two different empirical claims that the correlational metric cannot, by construction, distinguish. Only an intervention can.

7 sae-causal-audit: The Methodology as an Instrument

A finding locked inside its own experiment scripts is a claim, not a tool: anyone wanting *their* inert rate would have to separate methodology from experiment and re-implement the former. The

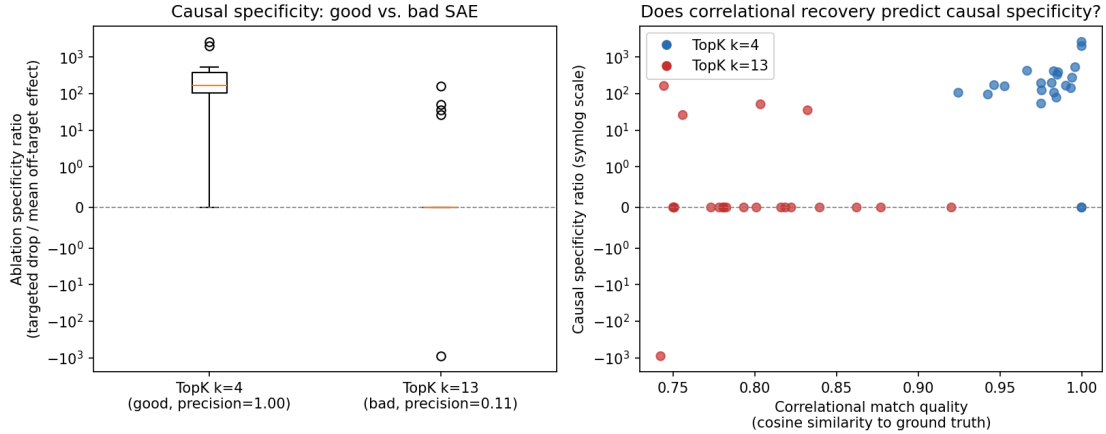


Figure 3: Left: ablation specificity by SAE (symlog scale; some values are exactly zero or negative). Right: cosine similarity vs. specificity, both SAEs pooled ($r = 0.657$).

methodology is therefore packaged as `sae-causal-audit`, an instrument that must survive inputs its author never anticipated. That constraint forced the design decisions below, surfaced two further bugs (Section 8), and refined the original finding itself (Section 10).

7.1 Structural typing: what “any SAE” means

The audit accepts anything satisfying a three-member runtime-checkable protocol—no inheritance, no registration, no adapter:

```
@runtime_checkable
class SparseAutoencoder(Protocol):
    W_dec: torch.Tensor # (d_sae, d_in)
    def encode(self, h: torch.Tensor) -> torch.Tensor: ...
    def decode(self, f: torch.Tensor) -> torch.Tensor: ...
```

`sae_lens.SAE`—the interface behind most published open-weight suites—satisfies it out of the box. The division of labor restates the central finding structurally: **matching uses `W_dec`** (the correlational step *should* use decoder geometry; that is what the field’s metric measures and what is being audited), while **every causal metric routes through `encode/decode`**, so encoder-side selection—the thing decoder geometry cannot certify—is what is actually measured. Ground truth enters through a second protocol (`FeatureProbe: activations_with_feature, activations_without_feature`), the seam that lets one pipeline serve two regimes: synthetic isolation inputs (exact ground truth) in the toy regime; positive/negative prompt sets captured at a hook point (a stated, weaker substitution) in the real-model regime of Section 11.

7.2 Three measurements per matched pair

In increasing order of causal strength, all propagated through `encode/decode` and a caller-supplied behavioral readout:

1. **Fired fraction** (the cheap screen that alone explained all seventeen inert cases of Section 6.3):

$$\text{fired_frac}_{i,j} = \frac{1}{N} \sum_{n=1}^N \mathbb{I}[|f_j(h_n)| > \varepsilon], \quad h_n \sim \text{feature } i \text{ present.} \quad (5)$$

If the encoder never activates atom j across hundreds of feature-present samples, no downstream claim about j “representing” i can be causal, whatever the cosine says. Cost: one batched encode.

2. **Ablation specificity** (read direction), as in Section 6.
3. **Sign-correct steering specificity** (write direction), with the match sign a required argument.

7.3 Interface design as bug prevention

The sign bug of Section 6 is made unrepresentable rather than merely fixed. The matcher returns both facts as separate fields instead of collapsing them into one lossy number:

```
@dataclass(frozen=True, slots=True)
class MatchResult:
    feature_idx: int
    atom_idx: int
    cosine: float # unsigned, in [0, 1] -- recovery semantics
    sign: float # +/-1.0 -- intervention semantics
```

and `steering_effect(...)` has no default sign to silently assume. The original failure is a permanent regression test: an anti-aligned atom steered with the wrong sign must read exactly 0.0 through a ReLU downstream, and with the correct sign must read positive.

Two further semantics rules are load-bearing. **Zeros carry their cause**: specificity 0.0 can mean “weak effect” or “the code was already zero,” with opposite implications; every causal result therefore ships with its `fired_frac`, and the `causally_inert` flag is *defined* as `fired_frac = 0.0`, never inferred from the ratio. **Infinity is legal where it means something and rejected where it does not**: a perfectly surgical intervention (nonzero target, exactly zero collateral) is legitimately infinite specificity; the serializer encodes it safely (strict JSON has no `Infinity` literal), median-based statistics accept it, and mean-based statistics refuse it loudly.

7.4 Test suite as instrument calibration

The suite (37 tests) works in three layers: *hand-derivable unit cases* (identity SAEs and two-atom constructions where every expected value is computable on paper: a perfectly surgical ablation returns exactly ∞ ; a structurally dead atom returns exactly 0.0 with `fired_frac = 0.0`; equal mixing into one other dimension returns specificity exactly 1.0—the same standard applied to D_i , where the antipodal pair must come out at exactly 0.5); *property-based tests* (Hypothesis) over randomized inputs (cosine stays in $[0, 1]$; the reported sign agrees with the true signed cosine of the chosen pair; dead atoms never win a match through a 0/0; bootstrap intervals are ordered and bracketed by the sample range); and an *end-to-end integration layer* that trains the real toy setting—real optimizer, real TopK competition—and asserts the audit recovers the known qualitative answers, with any shape mismatch failing loudly rather than producing numbers on misaligned tensors.

8 Reproducibility as a Stack of Claims

The package makes a specific, checkable promise: two runs producing identical science produce byte-identical result files, letting CI regenerate every result and compare SHA-256 hashes—regression detection on the *results*, not just the code. Actually executing that promise, rather than asserting it, surfaced two bugs (the third and fourth caught across this line of work) and forced the promise itself to be restated.

A serialization leak. The first attempted proof failed: two back-to-back from-scratch reproductions—same seeds, single-threaded, `torch.use_deterministic_algorithms(True)`—produced different hashes with identical scientific numbers. The report dataclass carried a `runtime_seconds` field; wall-clock timing was being serialized into the hashed JSON. The fix is a documented `VOLATILE_FIELDS` exclusion set: timing remains available in memory and in the human-readable render and never enters the hashed artifact. The gate is tested the only way a gate honestly can be: reproduce twice and require identical hashes, then deliberately corrupt one number and require the gate to fail, naming the file with expected and observed hashes.

Three layers of cross-environment nondeterminism. The same gate, run on CI, produced different hashes on different runs of the same commit—same seeds, same code, same pinned dependencies. Unwinding it went three layers deep:

1. **The optimizer was multi-threaded behind the scenes.** `torch.set_num_threads(1)` pins intra-op parallelism, but Adam defaults to `foreach=True`—a fused multi-tensor path whose floating-point accumulation order differs from the naive loop’s—and inter-op parallelism had never been pinned. Float addition is not associative; a different reduction order is a different last bit. Setting `foreach=False`, pinning interop threads to one, and adding defensive environment variables (`MKL_CBWR`, BLAS thread counts, `PYTHONHASHSEED`) restored single-environment determinism.
2. **The degraded SAE is a noise amplifier—a finding, not an inconvenience.** A last-bit difference should change a hash and nothing else; in the $k=13$ SAE it changed *integer counts*. One feature’s best-match cosine sits close enough to the 0.90 bar, and one atom’s pre-activation close enough to the TopK cut, that last-bit noise flips them across. The good SAE’s census never moved. The same TopK competition that produces causally inert matches also produces numerical boundary instability—one mechanism, two symptoms, at two different levels of the stack (Section 10 closes this loop). The repository encodes an explicit ± 1 tolerance band on that SAE’s counts, and only that SAE’s.
3. **“The same torch version” is not the same program.** With CI-side determinism proven, hashes still refused to match a baseline generated on Windows: the `torch==2.13.0+cpu` wheels for Windows and Linux are different binaries (MSVC vs. GCC 13.3; MKL 2026.1 vs. 2024.2; NNPACK off vs. on). Different compiled code rounds differently in the last bit—deterministically, permanently. No environment variable bridges it. **Byte-exact cross-platform reproducibility is not hard; it is unavailable, by construction.** The visible symptom was layer two in different clothes: the degraded SAE’s census read one feature higher on Windows while every continuous metric agreed to four decimal places.

The restructured promise. The repository now makes two guarantees with disjoint scopes. **Byte-exact** (SHA-256-identical result files) is guaranteed within the pinned CI environment

Table 5: The causal census over the 22 well-represented features, cosine ≥ 0.90 recovery bar, deterministic pipeline. Bracketed ranges are 95% bootstrap confidence intervals. [†]Medians and intervals are computed over every recovered pair across all 32 features ($n = 24$ and $n = 21$ respectively), not only the well-represented 22 used for the census rows.

	TopK $k=4$ (good)	TopK $k=13$ (bad)
Correlationally recovered, of 22	22 / 22	18 / 22
Recovered but causally inert (fired_frac = 0)	2 (9%)	3 (17%); 4 in ± 1 band
Median ablation specificity [†]	133.8 [107.0–167.4]	68.3 [22.0 –105.9]
Median steering specificity [†]	37.7 [33.1–41.8]	16.9 [14.2–21.3]

(ubuntu-24.04, torch==2.13.0+cpu) and verified on every push; the gate has produced identical hashes across three independent runs of one commit. **Semantic** (every number within $\text{rtol} = 10^{-4}$; boundary-sensitive counts within ± 1) is guaranteed on any platform. The byte-exact baseline is generated by the CI environment itself via a manually dispatched job—never regenerated locally—because a baseline produced on the wrong machine *was* the entire third layer. The tolerance tier is tested like the hash tier: corrupt a count by two, or nudge a continuous metric by one percent, and the gate must fail; both do. The generalization we propose: *reproducibility is not one claim but a stack of claims with different scopes, and an honest repository states which rung it guarantees where*. Most “fully reproducible” badges silently mean *on my machine, probably*.

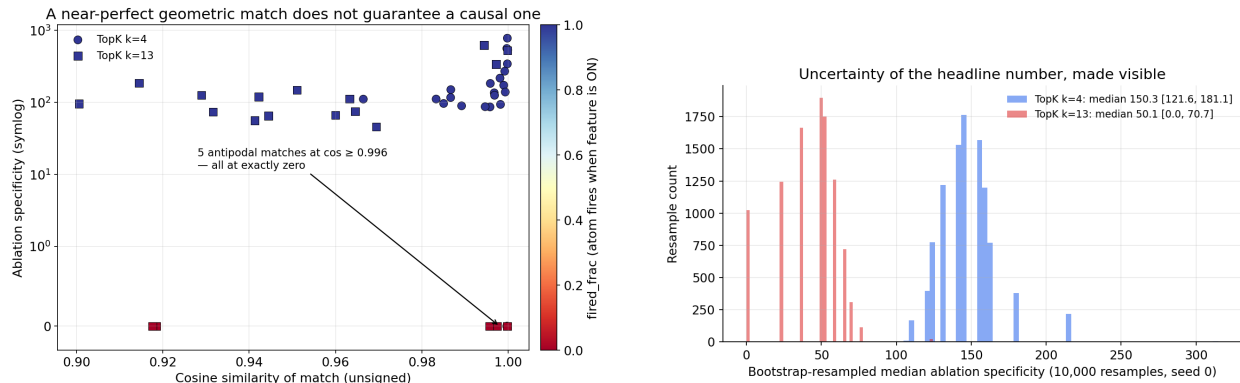
9 Re-audit Under the Deterministic Pipeline

The deterministic pipeline (single-threaded, `foreach=False`, deterministic algorithms enforced, fixed seeds; ~ 3 minutes on CPU) retrains the original configuration and audits the same two TopK SAEs ($k=4$, $k=13$; 128-atom dictionaries). One environmental note deserves a full sentence: the fully deterministic pipeline converges to *different weights* than the original multi-threaded runs, so the exact counts moved while the qualitative finding did not—precisely the non-transferability the original analysis warned about when it noted that a number measured for one checkpoint is not a portable constant. This is an accidental robustness check the claim passed. All numbers below come from the reference environment whose hashes CI verifies; where boundary sensitivity makes a count environment-dependent, it is stated at the number.

Three observations in Table 5 (Figure 4) deserve slow reading.

First, the inert features are not marginal matches—in either SAE. The bad SAE’s inert cosines are 0.986, 0.994, and 0.9995; the good SAE’s two are **0.9997** and **0.9998**. The *good* SAE—whose recovered set is otherwise causally excellent—contains features whose geometric match is perfect to three decimal places and whose atoms do not fire once across 500 feature-present samples. The original experiment’s most alarming single example was inert at cosine 0.92; the instrument, on a fresh run, finds inert matches at effectively 1.000, in both SAEs. The correlational metric is not merely noisy at the margin; it can be maximally confident and causally wrong, and SAE quality does not prevent it. Section 10 gives the clean geometric reason the good SAE, of all things, produces the most perfect inert matches.

Second, the correlational metric is least trustworthy exactly where it is most needed. Correlation between cosine and log ablation specificity, over each SAE’s *firing* pairs: $r = 0.859$ ($n = 21$) for the good SAE versus $r = 0.471$ ($n = 18$) for the bad one. The earlier pooled figure ($r = 0.657$) hid the decision-relevant shape: on a healthy dictionary cosine is a decent (never sufficient) proxy for causal quality; on a degraded dictionary—the case an audit exists to catch—the



(a) Cosine vs. ablation specificity (symlog), colored by `fired_frac`; high-cosine points span from specificity in the hundreds to exactly zero.

(b) 10,000 bootstrap-resampled medians per SAE: the good SAE’s distribution sits tightly; the bad SAE’s is wide, multimodal, with mass reaching toward zero.

Figure 4: The re-audit under the deterministic pipeline.

proxy itself degrades toward uninformative. Cosine similarity is a fair-weather instrument.

Third, the degraded SAE’s uncertainty structure is itself the finding. The good SAE’s 95% bootstrap interval on median ablation specificity spans 107–167 around 134 (relative width $\approx 45\%$); the bad SAE’s spans 22–106 around 68 (relative width **123%**), its lower bound within an order of magnitude of the specificity-1 collateral-damage line. A percentile bootstrap over an 18-pair recovered set containing three exact zeros produces resampled medians swinging across most of the observed range. Every summary number ships with a seeded, 10,000-resample interval [5]—the upgrade the original $n \approx 20$ point estimates explicitly owed.

The count carrying the ± 1 annotation is itself data: on the reference Linux environment the bad SAE’s census reads 18 recovered with 3 inert; on a Windows build of the same torch version it reads 19 with 4, because one non-antipodal feature (cosine 0.924) sits close enough to both the recovery bar and the TopK selection boundary that different BLAS builds resolve it differently. That flip-prone feature is the competitive-inertness case of the next section—the boundary sensitivity of Section 8 and the causal taxonomy below are the same phenomenon seen from two altitudes.

10 Anatomy of the Perfect Inert Matches

10.1 Antipodal pairs, end to end

The good SAE’s two inert features are the same geometric event twice. Feature 8 matches atom 78 at cosine 0.9997, *anti*-aligned (sign $-$), with `fired_frac` = 0.00: ablation specificity exactly 0. But atom 78 is simultaneously the best match for *feature 1*—same atom, positively aligned, firing on 100% of feature-1 samples with ablation specificity 535. One atom, two “recovered” features. The other inert feature is the identical story at different indices: feature 5 matches atom 22 at cosine 0.9998, anti-aligned, never fires, while feature 0 matches the same atom positively aligned, firing every time at specificity 347.

The toy model’s weights explain everything: $\cos(W_1, W_8) = -0.9998$ and $\cos(W_0, W_5) = -0.9999$. Both are **antipodal pairs**—the exact $W_i \approx -W_j$ configuration the superposition analysis predicts (Section 4)—two well-represented features sharing one direction with opposite signs. The SAE, reasonably, learned *one atom per shared axis*. The encoder’s TopK-then-ReLU selection passes only positive pre-activations, so each atom fires for the positive side of its axis and never for

the negative side. Decoder geometry for the negative-side feature is essentially perfect; encoder behavior for it is essentially nonexistent. This answers why the *good* SAE produces the most perfect inert matches: **antipodal inertness is not a training failure; it is superposition geometry faithfully compressed**. A better SAE learns the shared axis *more* precisely, pushing the anti-aligned cosine *closer* to 1.000. The failure is structural, and no amount of SAE quality removes it—only interface-level awareness of match sign does. The bad SAE replicates the pattern threefold (atoms 22, 21, 39 each serving pairs 0/5, 1/8, 4/2, each pair anti-parallel in ground truth to at least -0.999). Across both SAEs: **five antipodal pairs, accounting on the reference environment for every causally inert feature in both recovered sets**.

10.2 A two-axis taxonomy of causal inertness

By cause. *Structural inertness*—the antipodal mechanism—appears in good and bad SAEs alike, survives improved training, and is diagnosable from geometry alone: a shared atom matched with opposite signs is visible in the match table before any intervention runs. It is a hierarchy-free sibling of feature absorption [4]: both produce latents that do not fire where their concept is present and both resist being trained away, but absorption is driven by the sparsity objective over hierarchical features, while structural inertness needs nothing beyond an antipodal pair and a positive-pass encoder. *Competitive inertness*—an atom that keeps losing the TopK competition despite decent geometry, the dominant mechanism behind the original 77% figure—appears only in the degraded SAE. Its fourth inert case, the non-antipodal feature at cosine 0.924 that flips in and out of the census across BLAS builds, is this second kind, and the coincidence is not one: an atom that *barely* loses the TopK competition is an atom whose selection is decided in the last bits of a float. Competitive inertness and numerical boundary sensitivity are one mechanism expressed at two levels. Operationally the two kinds are opposites: structural inertness is stable, predictable, and screenable from the dictionary alone; competitive inertness is unstable by nature and is what the `fired_frac` screen exists to catch empirically.

By direction. The antipodal features are ablation-inert—one cannot remove what never activates—but they are *not* steering-inert: forcing atom 78 on *with the correct negative sign* raises feature 8’s reconstructed output at steering specificity **310**; the same intervention on feature 5’s atom scores 203; the bad SAE’s three antipodal cases score 143–261. These are among the highest steering specificities in the entire audit, attached to atoms whose ablation effect is exactly zero. The decoder direction is causally *usable* in the write direction even though the encoder never uses it in the read direction. “Causally inert” therefore decomposes into two separable claims—**read-inert** (the encoder never selects the atom for the feature: ablation-based monitoring built on it is blind) and **write-inert** (interventions along the atom do not move the feature: steering built on it is impotent)—and all five antipodal pairs dissociate the two completely. A feature can be unmonitorable yet steerable through the same atom; in this setting that is not an edge case but the *systematic signature* of antipodal superposition under a positive-pass encoder. This decomposition was invisible in the original experiment: without the signed-steering machinery working correctly, the write-direction test on anti-aligned matches returned zeros for the wrong reason. Fixing the instrument revealed structure the bug had been hiding.

Two design notes fall out of the same examples. The matcher is deliberately greedy (per-feature argmax) rather than a bipartite assignment, precisely so atom collisions—one atom best-matching two features—surface in the results instead of being optimized away; a collision is a finding about the dictionary (one atom serving an antipodal pair), not matching noise, and the audit’s five collisions were the thread that unraveled everything above. And the `fired_frac` screen would have flagged all five for the cost of one batched `encode`, before any intervention ran—at production scale, the

difference between a cheap census and an expensive one.

11 Toward Production SAEs: The Real-Model Harness

The package ships `scripts/audit_real_sae.py`: the identical pipeline pointed at published production SAEs—GPT-2 residual-stream SAEs and Gemma Scope [10]—via SAELens and TransformerLens. The real setting forces two substitutions, made explicit rather than smuggled in:

- **Ground truth** → **probe datasets**. “Feature present” is defined by positive/negative prompt sets per labeled concept, activations captured at the SAE’s hook point on the final token; the concept direction needed for cosine matching is a difference-in-means linear-probe direction—explicitly the weakest link in the real regime, a proxy standing in for ground truth that does not exist, and precisely why the pipeline is calibrated first in the toy regime where the test suite can assert the right answers.
- **Toy readout** → **spliced logits**. The downstream readout decodes the (possibly intervened) code, splices the reconstruction back into the residual stream at the hook point, runs the remainder of the model, and reads mean logits over concept-diagnostic tokens; ablation specificity then asks whether zeroing the atom suppresses *this* concept’s tokens more than it moves the other audited concepts’.

GPT-2-small runs on a free-tier T4 (or CPU, slowly); Gemma-2-2b with Gemma Scope wants ~16 GB of GPU memory. The research question the harness exists to answer is the obvious promotion of everything above: *of the features in a published production SAE that pass a standard correlational bar for a labeled concept, what fraction never fire when the concept is actually present—and among those that fire, how does read-specificity relate to write-specificity?* We do not know, and we specifically do not predict that toy-regime rates transfer—the probe-direction proxy alone guarantees the numbers are not comparable one-to-one. Real-model results from this harness should be reported at multiple cosine thresholds and multiple hook layers, never as a single number. That census is future work; the harness has not yet been executed against a production SAE as of this writing.

12 Practical Guidance

Given the measurements above, the responsible role for an SAE-derived signal in any deployed system is narrow: a *routing signal for closer review*, layered on top of—never substituting for—output-level safety filtering. Concretely: (i) never gate an automated action directly on raw SAE feature activation, or its absence—in one realistic configuration a majority (77%) of correlationally matched features were acting-on-noise candidates; (ii) any feature used for monitoring or steering needs its own causal validation pass *for the direction of use*—a monitoring use case needs read-validation, a steering use case needs write-validation, and this work measured five cases where one direction passes at specificity 143–310 while the other sits at exactly zero; (iii) re-run causal validation whenever the SAE is retrained: an atom’s causal relationship to a concept is a property of one trained checkpoint, not of the “feature” as a portable abstraction (the deterministic re-run of Section 9, which moved every exact count while preserving every qualitative claim, is a measured demonstration); and (iv) version-pin SAE checkpoints to the exact model checkpoint and data distribution they were validated against.

One epistemic note extends beyond deployment. The adversarial-interpretability concern raised in [1] is sharpened by these results: if decoder-geometry alignment diverges this far from encoder-

Table 6: Validation levels for SAE-feature claims, in increasing causal strength.

Validation level	Establishes	Cost	Still cannot tell you
Cosine match to a known/probed direction	Decoder-geometry alignment	Lowest	Whether the encoder ever selects the atom—this work found cosine ≈ 1.000 counterexamples in a well-trained SAE
<code>fired_frac</code> screen (one batched <code>encode</code>)	Whether the atom activates when the feature is present	Near zero	Whether the activation is <i>specific</i> ; an atom can fire promiscuously
Ablation specificity	Read-direction causal specificity: monitoring built on this atom sees the feature	Moderate	Write-direction behavior; out-of-distribution transfer
Sign-correct steering specificity	Write-direction causal specificity: interventions along this atom move the feature	Moderate	Read-direction behavior—five antipodal pairs dissociate the two completely

activation reality *without any adversarial pressure*—purely as an ordinary training-dynamics artifact—a model under actual incentive to obscure a mechanism has an even larger space of geometrically plausible, causally misleading solutions available than this work explores.

13 Limitations

Every quantitative claim here comes from small, fully synthetic toy models; real transformer residual streams involve orders of magnitude more features, genuine cross-layer and cross-token structure, and no accessible ground truth of the kind repeatedly relied on above. The toy setting is what makes rigorous validation possible, and exactly what makes every absolute number a calibration point for the easy case. The causal battery tests features in isolation on the ON side; interaction effects among simultaneously active features are untested in both regimes. The read/write decomposition rests on five antipodal pairs across two SAEs—systematic within this run—but within one training run and one seed; the structural/competitive split likewise rests on one run and its ± 1 boundary band, not a rate measured across seeds. Sample sizes remain small (22 well-represented features, 2 SAE configurations, 1 seed per SAE under the deterministic pipeline; the bootstrap quantifies within-sample uncertainty, not across-seed variance, which the earlier 3-seed protocol measured and the tool supports re-running). Exact specificity values (168.8, 133.8, ...) should be read as the shape of an effect measured once on this model, not tight estimates; the reproducible claim is qualitative (correlational match quality is informative but leaves a large, characterizable causal gap). The difference-in-means probe direction of the real harness is a linear approximation standing in for absent ground truth, and realistic prompt sets undersample a concept’s distribution—both weaken the correlational side of a real-regime audit, while the causal side (`fired_frac`, ablation, steering) does not depend on the direction estimate. Finally, four bugs caught and reported across this line of work is not evidence the pipeline is now bug-free; it is evidence the discipline that catches them is working. A previous draft of this list predicted a fourth existed, and it did; the expected steady state is that a fifth does too.

14 Conclusion

Three claims were checked by experiment rather than asserted, and each held with a real, quantified complication attached. Sparse features are packed non-orthogonally into a smaller space when sparsity makes it cheap, in a geometry precise enough that a convergence bug was catchable by comparison against the theoretically required answer—and precise enough to hide a third, diffuse-sharing regime behind a metric’s blind spot. An overcomplete sparse autoencoder can partially undo the packing, with TopK dominating L_1 for a mechanistically identified reason (shrinkage) confirmed by direct magnitude-refinement measurement, not by unexplained superiority. And a geometric recovery, even a nearly perfect one, is not a causal one, by a margin measured directly: up to 77% of correlationally recovered features causally inert in a degraded SAE, 9% in an excellent one, with inert matches at cosine ≈ 1.000 .

Turning the finding into an instrument complicated it productively, twice. Causal inertness decomposes by cause—structural inertness generated systematically by antipodal superposition geometry under a positive-pass encoder, present in good SAEs and removable only at the interface level; competitive inertness, a selection pathology of degraded dictionaries that is simultaneously the locus of their numerical instability—and by direction, with read-inertness and write-inertness dissociating completely on every antipodal pair measured: features perfectly matched, completely unmonitorable, and highly steerable, all at once, through one atom apiece. And executing the reproducibility claim, rather than asserting it, established that byte-exact reproduction is a single-environment guarantee by construction, and that honest repositories should state which rung of the reproducibility stack they guarantee where.

The instrument is public: `make reproduce` regenerates every number here in about three minutes on CPU; `make verify` checks them within tolerance on any machine; `make verify-hashes` holds the pinned CI environment to the byte; and the protocol interface accepts any SAE that can `encode`, `decode`, and show its dictionary. The corresponding census over published production SAEs—in both causal directions, at multiple thresholds and hook layers—is the immediate next step.

Reproducibility statement. All code, the 37-test suite (including property-based tests), the deterministic reproduction pipeline, the hash-verified CI gate, and the real-model harness are available at <https://github.com/mohamed-bal/sae-causal-audit>; the toy-model and SAE reproduction code is at <https://github.com/mohamed-bal/superposition-to-monosemanticity>. Every number in this paper is written by code to a `results/*.json` file whose SHA-256 hash is committed and verified in CI; every figure is regenerated by re-running the corresponding script.

References

- [1] Leonard Bereska and Efstratios Gavves. Mechanistic interpretability for ai safety — a review. *arXiv preprint arXiv:2404.14082*, 2024.
- [2] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.

- [3] Bart Bussmann, Patrick Leask, and Neel Nanda. Batchtopk sparse autoencoders. *arXiv preprint arXiv:2412.06410*, 2024.
- [4] David Chanin, James Wilken-Smith, Tomáš Dulka, Hardik Bhatnagar, Satvik Golechha, and Joseph Bloom. A is for absorption: Studying feature splitting and absorption in sparse autoencoders. *arXiv preprint arXiv:2409.14507*, 2024.
- [5] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
- [6] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/toy_model/index.html.
- [7] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- [8] William B. Johnson and Joram Lindenstrauss. Extensions of lipschitz mappings into a hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.
- [9] Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Bloom, David Chanin, Yeu-Tong Lau, Eoin Farrell, Callum McDougall, Kola Ayonrinde, Demian Till, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. Saebench: A comprehensive benchmark for sparse autoencoders in language model interpretability. *arXiv preprint arXiv:2503.09532*, 2025.
- [10] Tom Lieberum, Senthooan Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2. *arXiv preprint arXiv:2408.05147*, 2024.
- [11] Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, 1996.
- [12] Senthooan Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah, and Neel Nanda. Improving dictionary learning with gated sparse autoencoders. *arXiv preprint arXiv:2404.16014*, 2024.
- [13] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermyn, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*, 2024. <https://transformer-circuits.pub/2024/scaling-monosemanticity/>.
- [14] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996.